

Task 5 — : Upload and Retrieve Application Files Using AWS CLI

Intern: Nidhi Pandey

Program: Evolixa AWS Cloud Internship

Date: 2026-09-05

1. Approach and Architecture

The objective of this task was to use the AWS CLI to automate S3 file operations instead of performing every operation manually through the AWS Console. The goal was to move beyond the click-ops approach and develop a basic programmatic workflow for handling files in the cloud.

I took a hands-on approach where I started with the basics: configuring the AWS CLI securely, creating a local project directory, and then performing a series of operations like uploading, listing, downloading, and synchronizing files with S3. I tested how the system handles modified files and ensured I understood the underlying commands. The architecture is simple: it's a local environment interacting directly with S3 via the CLI. The key learning was to rely on IAM profiles for authentication instead of hardcoding credentials, and to use the sync command for efficient file transfer. I also created a dedicated S3 bucket to keep the test data organized and isolated from other projects.

2. Implementation Steps

Step	Action	Command
1	Configure AWS CLI	<code>aws configure --profile s3-admin</code>
2	Verify Identity	<code>aws sts get-caller-identity --profile s3-admin</code>
3	Create S3 Bucket	<code>aws s3 mb s3://task5-cli-bucket --region us-east-1 --profile s3-</code>

Step	Action	Command
4	Create Local Directory	<code>mkdir task5-cli-files</code>
5	Upload Individual Files	<code>aws s3 cp file1.txt s3://task5-cli-bucket/ --profile s3-admin</code>
6	Upload a Directory	<code>aws s3 cp backups/ s3://task5-cli-bucket/backups/ --recursive --profile s3-admin</code>
7	List Stored Objects	<code>aws s3 ls s3://task5-cli-bucket/ --recursive --profile s3-admin</code>
8	Download Selected Objects	<code>aws s3 cp s3://task5-cli-bucket/file1.txt ./downloaded-file1.txt --profile s3-admin</code>
9	Synchronize Local Directory	<code>aws s3 sync ./ s3://task5-cli-bucket/ --profile s3-admin</code>
10	Test Modified Files	Modified file1.txt, ran <code>aws s3 sync ./ s3://task5-cli-bucket/ --profile s3-admin</code>
11	Document Commands	Listed all commands in README
12	Explain Security	Documented why credentials shouldn't be hardcoded
13	Identify Automation	Suggested using cron jobs or Lambda
14	Test Clean Directory	Downloaded all files to a new folder
15	Remove Test Data	<code>aws s3 rm s3://task5-cli-bucket/ --recursive --profile s3-admin</code> and <code>aws s3 rb s3://task5-cli-bucket/ --profile s3-admin</code>

3. Key AWS Concepts Learned

- **AWS CLI Profiles:** Using named profiles (like s3-admin) allows you to easily switch between different users/roles without changing your credentials.
 - **S3 Sync:** The aws s3 sync command is extremely efficient; it only uploads files that have been added or modified, saving time and bandwidth.
 - **Recursive Operations:** The --recursive flag is crucial for uploading or downloading entire directories.
 - **S3 Object Listing:** aws s3 ls with --recursive shows all objects in the bucket, which is helpful for verifying uploads.
 - **Idempotent Operations:** Commands like sync are idempotent; running them multiple times won't cause duplicate uploads.
 - **Least Privilege & Profiles:** Using a dedicated s3-admin profile ensures that I was using the right set of permissions for S3 operations, preventing accidental mismatches with other users like sns-user.
-

4. Challenges Faced

1. **Profile Mismatch:** When I first ran the aws s3 cp command, I kept getting AccessDenied errors. This was because my default CLI profile was sns-user, which did not have any S3 permissions. I didn't realize this initially and spent time checking the bucket's permissions.
 2. **Missing --recursive Flag:** When I tried to upload the backups/ directory, I forgot to add the --recursive flag, which resulted in an error saying the path didn't exist.
 3. **Modified File Sync:** While testing how modified files are handled, I initially couldn't see the change in S3 because I forgot that sync only uploads files with a different file size or modification time. I had to confirm that the local file was indeed saved.
-

5. Solutions Implemented

1. **Profile Fix:** I resolved the AccessDenied errors by appending --profile s3-admin to all my CLI commands. This ensured I was using the IAM user with the correct S3 permissions.
2. **CLI Syntax Fix:** I corrected my directory upload command by adding the --recursive flag, which uploaded all files within the backups/ directory to the backups/ prefix in S3.

3. **Sync Verification:** I used `ls -l` locally to check the modification time of files before running the sync command. This helped me understand that sync works based on file size and timestamp.

6. Security Considerations

- **Credentials:** I never hardcoded any Access Keys or Secret Keys in the scripts or the README. All authentication was handled by the AWS CLI using the `s3-admin` profile, which stores credentials securely.
- **Profile Management:** Using profiles ensures that I am not accidentally using administrative-level permissions for routine tasks.
- **Bucket Cleanup:** I ensured that all test data was deleted after the task to avoid unnecessary costs and prevent leaving unused data in the cloud.
- **No Public Access:** I kept the bucket private and only accessed it via the CLI, which is a secure method of interaction.

7. Results and Observations

Allowed Operations

Operation	Command	Result
Upload	<code>aws s3 cp file1.txt s3://task5-cli-bucket/</code>	Uploaded
Directory Upload	<code>aws s3 cp backups/ s3://task5-cli-bucket/backups/ --recursive</code>	Uploaded
List	<code>aws s3 ls s3://task5-cli-bucket/ --recursive</code>	Listed all files
Download	<code>aws s3 cp s3://task5-cli-bucket/file1.txt ./downloaded-file1.txt</code>	Downloaded
Sync	<code>aws s3 sync ./ s3://task5-cli-bucket/</code>	Synced new/mod files

Observations

- The CLI is **significantly faster** than the console for batch operations.

- The sync command is a powerful tool for maintaining consistency between local and cloud storage, as it automatically handles file changes.
- Using profiles is essential for multi-user or multi-role scenarios.

8. Possible Improvements for Production

Improvement	Benefit
Use IAM Roles (for EC2)	If this workflow runs on an EC2 instance, attach an IAM role to the instance instead of using access keys.
Enable S3 Versioning	Protect against accidental overwrites.
Use Lifecycle Policies	Automatically transition older backups to cheaper storage (Glacier).
Automate with Cron/Lambda	Schedule the sync command to run daily for backups using a cron job or AWS Lambda.
Enable S3 Server Access Logging	Track all requests made to the bucket for auditing.

Conclusion

This task demonstrated the power and efficiency of the AWS CLI for automating cloud file operations. By using simple commands, I was able to upload, download, and synchronize files quickly. The key takeaway is that for repetitive tasks, the CLI offers a significant speed advantage over the console, and when combined with IAM profiles and proper cleanup, it's a robust solution for scripting and automation.